

Scenario 1 – Amazon-style Electronics Store

Aim

To create an e-commerce homepage for an electronics store with product categories, a promotional banner, product cards, prices, and Add to Cart functionality.

Algorithm

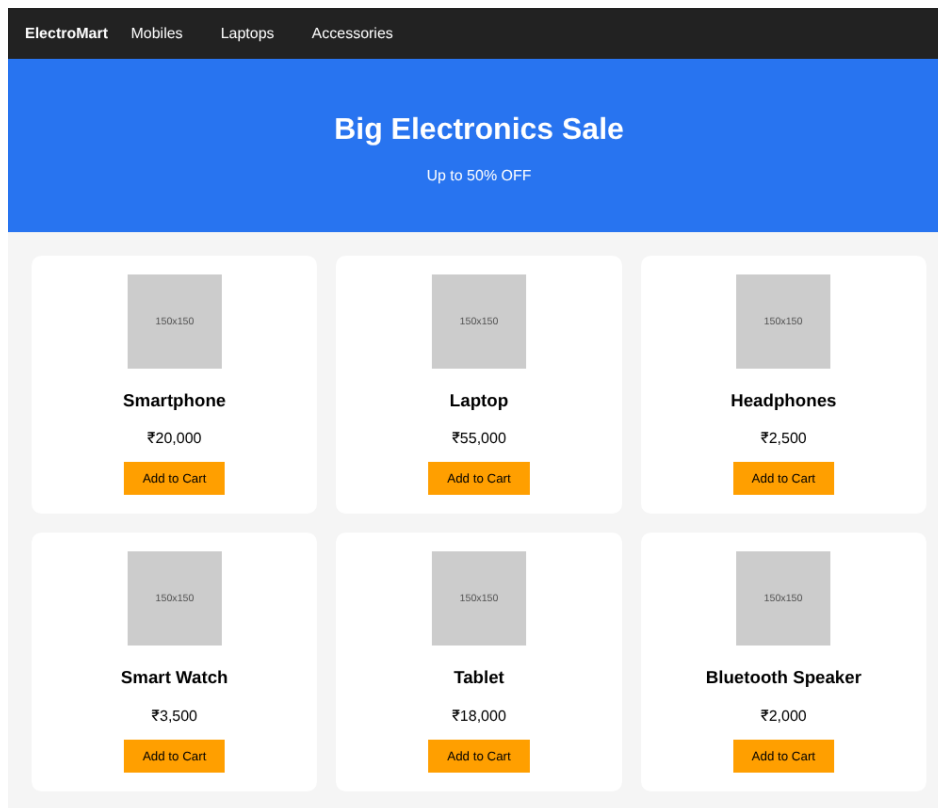
1. Start the program and create the basic HTML document structure.
2. Set the page title and define CSS styles for the navigation bar, banner, product grid, product cards, images, and buttons.
3. Create a navigation bar containing the store name and product categories such as Mobiles, Laptops, and Accessories.
4. Create a promotional banner that displays a sale message or discount.
5. Declare a JavaScript array containing at least six products and their prices.
6. Create an empty container in the HTML where the product cards will be displayed.
7. Use a JavaScript loop or array mapping method to read each product from the array.
8. For each product, generate a card containing its image, product name, price, and Add to Cart button.
9. Attach a click event to each Add to Cart button so that the selected product is identified.
10. Display a confirmation message when a product is added, then open the page in a browser and test the buttons.

Source Code

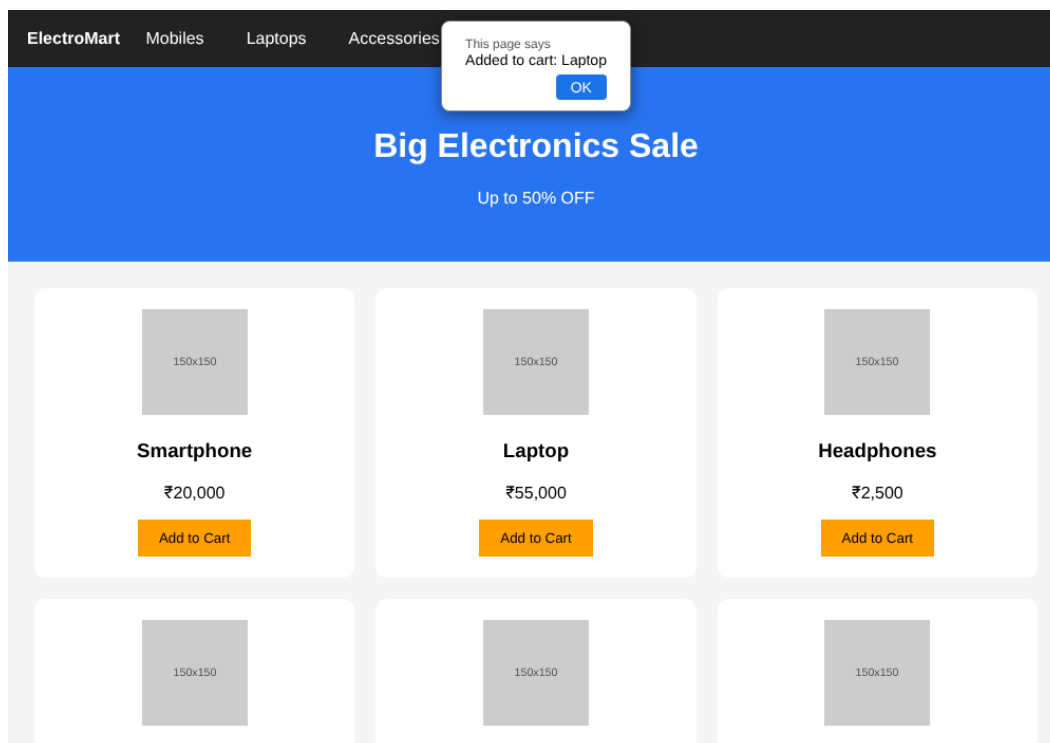
```
<!DOCTYPE html>
<html>
<head>
<title>Electronics Store</title>
<style>
body{font-family:Arial;margin:0;background:#f5f5f5}
nav{background:#222;color:white;padding:18px}
nav a{color:white;margin:20px;text-decoration:none}
.banner{background:#2874f0;color:white;text-align:center;padding:35px}
.products{display:grid;grid-template-columns:repeat(3,1fr);gap:20px;padding:25px}
.card{background:white;padding:20px;text-align:center;border-radius:10px}
.card img{width:120px;height:100px;object-fit:contain}
button{background:#ff9f00;border:0;padding:10px 20px;cursor:pointer}
</style>
</head>
<body>
<nav><b>ElectroMart</b>
<a href="#">Mobiles</a><a href="#">Laptops</a><a href="#">Accessories</a></nav>
<div class="banner"><h1>Big Electronics Sale</h1><p>Up to 50% OFF</p></div>
<div class="products" id="products"></div>
<script>
const items=[
["Smartphone","₹20,000"],["Laptop","₹55,000"],["Headphones","₹2,500"],
```

```
["Smart  
Watch", "₹3,500"], ["Tablet", "₹18,000"], ["Bluetooth  
Speaker", "₹2,000"]  
];  
document.getElementById("products").innerHTML=items.m  
ap(p=>`  
<div class="card">  
<h3>${p[0]}</h3><p>${p[1]}</p>  
<button onclick="alert('Added to cart: ${p[0]}')">Add  
to Cart</button></div>`).join("");  
</script>  
</body>  
</html>
```

Output



Output 1: Electronics store homepage with navigation bar, sale banner and six product cards



Output 2: Clicking "Add to Cart" on the Laptop card shows the confirmation message

Result

The homepage rendered with the navigation bar, sale banner and six product cards (image, name, price). Each Add to Cart button identified its product and displayed a confirmation message.

Scenario 2 – Zomato-style Food Ordering Site

Aim

To develop a restaurant listing webpage with restaurant search, cuisine filtering, ratings, and a menu showing dishes with prices.

Algorithm

1. Start the program and create the HTML structure for the food-ordering page.
2. Add a heading area with the website name and a short description.
3. Create a search input to enter a restaurant name.
4. Create a cuisine drop-down list with options such as Indian, Chinese, and Italian.
5. Prepare a JavaScript array containing restaurant names, cuisine types, and ratings.
6. Create a container to display the restaurant results.
7. When the page loads, call a display function to show the available restaurants.
8. Inside the function, read the search text and selected cuisine from the form controls.
9. Filter the restaurant array by checking the name and cuisine conditions.
10. Display a card for every matching restaurant with its cuisine, rating, and at least five menu dishes with prices.
11. Call the display function whenever the search text or cuisine selection changes.
12. Test the page using different restaurant names and cuisine selections.

Source Code

```
<!DOCTYPE html>
<html>
<head>
<title>Food Ordering</title>
<style>
body{font-family:Arial;background:#f5f5f5;margin:0}
header{background:#1976d2;color:white;padding:20px;text-align:center}
.controls{text-align:center;padding:20px}
input,select{padding:10px;margin:5px}
.card{background:white;padding:20px;margin:15px;border-radius:10px}
</style></head>
<body>
<header><h1>Food Hub</h1><p>Find your favourite restaurant</p></header>
<div class="controls">
<input id="search" placeholder="Search restaurant">
<select id="cuisine"><option value="">All Cuisine</option>
<option>Indian</option><option>Chinese</option><option>Italian</option></select>
</div>
<div id="restaurants"></div>
<script>
const data=[
{name:"Spice Garden",cuisine:"Indian",rating:4.5},
{name:"Chinese Wok",cuisine:"Chinese",rating:4.2},
{name:"Pizza House",cuisine:"Italian",rating:4.6},
{name:"South Kitchen",cuisine:"Indian",rating:4.3}
];
function display(){
```

```
const
s=document.getElementById("search").value.toLowerCase
();
const c=document.getElementById("cuisine").value;
document.getElementById("restaurants").innerHTML=data
.filter(x=>
x.name.toLowerCase().includes(s)&&(!c||x.cuisine===c)
).map(x=>`
<div class="card"><h2>${x.name}</h2><p>Cuisine:
${x.cuisine}</p>
<p>Rating: ${x.rating}</p><h3>Menu</h3>
<p>Biriyani - ₹180 | Fried Rice - ₹150 | Noodles -
₹140</p>
<p>Pizza - ₹250 | Ice Cream -
₹80</p></div>`).join("");
}
document.getElementById("search").addEventListener("i
nput",display);
document.getElementById("cuisine").addEventListener("
change",display);
display();
</script></body></html>
```

Output

Food Hub

Find your favourite restaurant

All Cuisine

Spice Garden

Cuisine: Indian

Rating: 4.5

Menu

Biryani - ₹180 | Fried Rice - ₹150 | Noodles - ₹140

Pizza - ₹250 | Ice Cream - ₹80

Chinese Wok

Cuisine: Chinese

Rating: 4.2

Menu

Biryani - ₹180 | Fried Rice - ₹150 | Noodles - ₹140

Pizza - ₹250 | Ice Cream - ₹80

Pizza House

Cuisine: Italian

Rating: 4.6

Menu

Biryani - ₹180 | Fried Rice - ₹150 | Noodles - ₹140

Pizza - ₹250 | Ice Cream - ₹80

South Kitchen

Cuisine: Indian

Rating: 4.3

Menu

Biryani - ₹180 | Fried Rice - ₹150 | Noodles - ₹140

Pizza - ₹250 | Ice Cream - ₹80

Output 1: All restaurants displayed on page load (no filter)

Food Hub

Find your favourite restaurant

All Cuisine

Pizza House

Cuisine: Italian

Rating: 4.6

Menu

Biryani - ₹180 | Fried Rice - ₹150 | Noodles - ₹140

Pizza - ₹250 | Ice Cream - ₹80

Output 2: Searching "pizza" shows only Pizza House

Food Hub

Find your favourite restaurant

Search restaurant

Indian

Spice Garden

Cuisine: Indian

Rating: 4.5

Menu

Biryani - ₹180 | Fried Rice - ₹150 | Noodles - ₹140

Pizza - ₹250 | Ice Cream - ₹80

South Kitchen

Cuisine: Indian

Rating: 4.3

Menu

Biryani - ₹180 | Fried Rice - ₹150 | Noodles - ₹140

Pizza - ₹250 | Ice Cream - ₹80

Output 3: Cuisine filter "Indian" shows Spice Garden and South Kitchen

Food Hub

Find your favourite restaurant

xyz

Indian

Output 4: A name with no match (xyz) shows no restaurants

Result

The page listed all restaurants on load, and the list updated instantly with the search text and cuisine selection. Non-matching searches showed no cards.

Scenario 3 – Myntra-style Fashion Portal

Aim

To create a user registration form with JavaScript validation for email, password, mobile number, and confirm password.

Algorithm

1. Start the program and create an HTML registration form.
2. Add input fields for the user's name, email address, mobile number, password, and confirm password.
3. Set appropriate input types and required attributes for the form fields.
4. Create a message area to show validation errors.
5. When the user submits the form, call a JavaScript validation function.
6. Read the values entered in all the input fields.
7. Check the email address against a valid email pattern.
8. Check that the password has at least eight characters, one digit, and one special character.
9. Check that the mobile number contains exactly ten digits.
10. Compare the password and confirm-password values to ensure they match.
11. If any validation fails, show a suitable error message and stop submission.
12. If all checks pass, display a registration success message.

Source Code

```
<!DOCTYPE html>
<html>
<head><title>Fashion Registration</title>
<style>
body{text-align:center;font-
family:Arial;background:#f5f5f5}
form{background:white;width:330px;margin:40px
auto;padding:25px;border-radius:10px}
input{box-sizing:border-
box;display:block;width:100%;padding:10px;margin:10px
0}
button{padding:10px
30px;background:#ff3f6c;color:white;border:0}
#error{color:red}
</style></head>
<body>
<form onsubmit="return validate()">
<h2>Fashion Portal</h2>
<input id="name" placeholder="Name" required>
<input id="email" type="email" placeholder="Email"
required>
<input id="mobile" placeholder="Mobile Number"
required>
<input id="pass" type="password"
placeholder="Password" required>
<input id="confirm" type="password"
placeholder="Confirm Password" required>
<p id="error"></p><button
type="submit">Register</button>
</form>
<script>
function validate(){
```

```
const
email=document.getElementById("email").value.trim();
const
mobile=document.getElementById("mobile").value.trim()
;
const pass=document.getElementById("pass").value;
const
confirm=document.getElementById("confirm").value;
const error=document.getElementById("error");
if(!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)){
error.textContent="Enter a valid email";return
false;}
if(!/^(?=.*\d)(?=.*[A-Za-z0-9]).{8,}$/.test(pass)){
error.textContent="Password needs 8 characters, a
number and a special character";return false;}
if(!/^\\d{10}$/.test(mobile)){
error.textContent="Enter a valid 10-digit mobile
number";return false;}
if(pass!==confirm){error.textContent="Passwords do
not match";return false;}
alert("Registration successful");return false;
}
</script></body></html>
```

Output

Fashion Portal

Enter a valid email

Register

Output 1: Invalid email (ravi@gmail) shows an error

Fashion Portal

Password needs 8 characters, a number and a special character

Register

Output 2: Weak password (no digit/special character) shows an error

Fashion Portal

Enter a valid 10-digit mobile number

Register

Output 3: Mobile number with fewer than 10 digits shows an error

Fashion Portal

Passwords do not match

Register

Output 4: Mismatched confirm password shows an error

The screenshot shows a web form titled "Fashion Portal" for registration. At the top, a modal message box states "This page says Registration successful" with an "OK" button. Below the title, there are five input fields: a text field containing "Ravi", an email field containing "ravi@gmail.com", a text field containing "9876543210", and two password fields, both containing "*****". A red "Register" button is positioned at the bottom of the form.

Output 5: All inputs valid - "Registration successful" message

Result

Each invalid input (email, weak password, short mobile number, mismatched passwords) was rejected with a suitable error message; valid data displayed 'Registration successful'.

Scenario 4 – BookMyShow-style Ticket Booking

Aim

To create a PHP movie-ticket booking form that validates show timing and seat number, prevents duplicate seat selection within the session, and generates a booking ID.

Algorithm

1. Start the PHP program and call `session_start()` to enable session storage.
2. Initialize a session array to store seat numbers already booked during the current session.
3. Create an HTML form with a show-time selector and a seat-number field.

4. When the form is submitted, read the selected show time and seat number.
5. Check that the show time belongs to the list of valid show times.
6. Check that the seat number is within the allowed range of 1 to 100.
7. Search the session's booked-seat array to see whether that seat has already been booked.
8. If the input is invalid or the seat is already booked, prepare an appropriate error message.
9. If the seat is available, add its number to the session array.
10. Generate a unique booking ID for the successful booking.
11. Display the booking ID, selected show time, seat number, or error message on the page.
12. Run the PHP file through a PHP-enabled local server and test both valid and duplicate seat bookings.

Source Code

```
<?php
session_start();
if (!isset($_SESSION['booked'])) $_SESSION['booked']
= [];
$message = "";
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $time = $_POST["time"] ?? "";
    $seat = (int)($_POST["seat"] ?? 0);
    $validTimes = ["10:00 AM", "2:00 PM", "6:00 PM",
"9:00 PM"];
    if (!in_array($time, $validTimes, true)) {
        $message = "Please select a valid show
time.";
    } elseif ($seat < 1 || $seat > 100) {
        $message = "Seat number must be between 1 and
100.";
    } elseif (in_array($seat, $_SESSION["booked"],
true)) {
        $message = "That seat is already booked in
this session.";
    } else {
        $_SESSION["booked"][] = $seat;
        $id = "BK" .
strtoupper(bin2hex(random_bytes(3)));
        $message = "Booking successful! Booking ID: "
. $id .
        " | Show: " .
htmlspecialchars($time) . " | Seat: " . $seat;
    }
}
?>
<!DOCTYPE html>
<html><body>
```

```
<h2>Movie Ticket Booking</h2>
<form method="post">
<label>Show time:
<select name="time">
<option>10:00 AM</option><option>2:00 PM</option>
<option>6:00 PM</option><option>9:00 PM</option>
</select></label><br><br>
<label>Seat number (1-100): <input type="number"
name="seat" min="1" max="100"
required></label><br><br>
<button type="submit">Book Ticket</button>
</form>
<p><?php echo htmlspecialchars($message); ?></p>
</body></html>
```

Output

Movie Ticket Booking

Show time:

Seat number (1–100):

Output 1: Booking form on first load

Movie Ticket Booking

Show time:

Seat number (1–100):

Booking successful! Booking ID: BK9D8219 | Show: 6:00 PM | Seat: 25

Output 2: Valid booking (6:00 PM, seat 25) generates a Booking ID

Movie Ticket Booking

Show time:

Seat number (1–100):

That seat is already booked in this session.

Output 3: Booking seat 25 again in the same session shows the duplicate-seat error

Movie Ticket Booking

Show time:

Seat number (1–100):

Seat number must be between 1 and 100.

Output 4: Seat number 150 (outside 1-100) shows the range error

Result

A valid show time and seat generated a unique Booking ID. Re-booking the same seat in the same session and an out-of-range seat number were both rejected with error messages.

Scenario 5 – Flipkart-style Marketplace

Aim

To create a product catalog with product search and sorting by price or rating using JavaScript.

Algorithm

1. Start the program and create the HTML elements for search, sorting, and product display.
2. Prepare a JavaScript array containing product names, prices, and ratings.
3. Create a search input that accepts part or all of a product name.
4. Create a sorting drop-down with default order, price, and rating options.
5. Write a display function that reads the current search and sorting values.
6. Filter the product array so that only product names matching the search text remain.
7. If price sorting is selected, arrange the filtered products from lowest price to highest price.
8. If rating sorting is selected, arrange the filtered products from highest rating to lowest rating.

9. Clear the previous product display before rendering new results.
10. Display each matching product's name, price, and rating.
11. Call the display function when the search input or sorting option changes.
12. Test search terms and both sorting options to confirm the output updates correctly.

Source Code

```
<!DOCTYPE html>
<html><body>
<h2>Marketplace</h2>
<input id="search" placeholder="Search product">
<select id="sort"><option value="">Default
order</option>
<option value="price">Price: low to high</option>
<option value="rating">Rating: high to
low</option></select>
<div id="list"></div>
<script>
const products=[
{name:"Phone",price:20000,rating:4.5},
{name:"Laptop",price:55000,rating:4.8},
{name:"Headphone",price:2500,rating:4.2},
{name:"Watch",price:3500,rating:4.6}
];
function show(){
const
s=document.getElementById("search").value.toLowerCase
();
const sort=document.getElementById("sort").value;
let
result=products.filter(p=>p.name.toLowerCase().includ
es(s));
if(sort==="price") result.sort((a,b)=>a.price-
b.price);
if(sort==="rating") result.sort((a,b)=>b.rating-
a.rating);
document.getElementById("list").innerHTML=result.map(
p=>
`<div><h3>${p.name}</h3><p>Price: ₹${p.price} |
Rating: ${p.rating}</p></div>`
```

```
).join("") || "No products found.";
}
document.getElementById("search").addEventListener("input", show);
document.getElementById("sort").addEventListener("change", show);
show();
</script></body></html>
```

Output

Marketplace

Default order ▼

Phone

Price: ₹20000 | Rating: 4.5

Laptop

Price: ₹55000 | Rating: 4.8

Headphone

Price: ₹2500 | Rating: 4.2

Watch

Price: ₹3500 | Rating: 4.6

Output 1: Default order - all four products

Marketplace

Price: low to high ▼

Headphone

Price: ₹2500 | Rating: 4.2

Watch

Price: ₹3500 | Rating: 4.6

Phone

Price: ₹20000 | Rating: 4.5

Laptop

Price: ₹55000 | Rating: 4.8

Output 2: Sorted by price (low to high)

Marketplace

Rating: high to low ▼

Laptop

Price: ₹55000 | Rating: 4.8

Watch

Price: ₹3500 | Rating: 4.6

Phone

Price: ₹20000 | Rating: 4.5

Headphone

Price: ₹2500 | Rating: 4.2

Output 3: Sorted by rating (high to low)

Marketplace

Default order ▼

Phone

Price: ₹20000 | Rating: 4.5

Headphone

Price: ₹2500 | Rating: 4.2

Output 4: Searching "ph" shows Phone and Headphone

Marketplace

No products found.

Output 5: No match shows "No products found."

Result

Search filtered products by name, and sorting worked correctly: price from lowest to highest and rating from highest to lowest. No match displayed 'No products found.'

Scenario 6 – Swiggy-style Grocery App

Aim

To develop a grocery search module that displays suggestions as the user types and provides Add to Cart functionality.

Algorithm

1. Start the program and create a search input field.
2. Prepare a list of grocery items with their prices.
3. Create separate page areas for search suggestions and product results.
4. Attach an input event listener to the search field.
5. Whenever the user types, convert the entered text to lowercase.
6. Compare the search text with each grocery item's name.
7. Collect the items whose names contain the entered text.
8. Display the matching item names as search suggestions.
9. Display each matching item with its price and an Add to Cart button.

10. When the button is clicked, show a message identifying the selected item.
11. If no items match, display a suitable no-results message.
12. Test with different terms, including a blank search and a term with no matches.

Source Code

```
<!DOCTYPE html>
<html><body>
<h2>Grocery Store</h2>
<input id="search" placeholder="Search grocery">
<div id="suggestions"></div><div id="products"></div>
<script>
const items=[["Apple",120],["Banana",60],["Milk",50],
["Rice",80],["Bread",45],["Tomato",40]];
function searchItems(){
const
value=document.getElementById("search").value.toLowerCase();
const
result=items.filter(x=>x[0].toLowerCase().includes(value));
document.getElementById("suggestions").innerHTML=
result.map(x=><p>${x[0]}</p>).join("") || "No
suggestions found.";
document.getElementById("products").innerHTML=result.
map((x,i)=>`
<div><h3>${x[0]}</h3><p>₹${x[1]}</p>
<button onclick="addToCart('${x[0]}')">Add to
Cart</button></div>` ).join("");
}
function addToCart(name){alert(name+" added to
cart");}
document.getElementById("search").addEventListener("input",searchItems);
searchItems();
</script></body></html>
```

Output

Grocery Store

Apple

Banana

Milk

Rice

Bread

Tomato

Apple

₹120

Add to Cart

Banana

₹60

Add to Cart

Milk

₹50

Add to Cart

Rice

₹80

Add to Cart

Bread

₹45

Add to Cart

Tomato

₹40

Add to Cart

Output 1: Blank search shows all items as suggestions and products

Grocery Store

Apple

Banana

Bread

Tomato

Apple

₹120

Add to Cart

Banana

₹60

Add to Cart

Bread

₹45

Add to Cart

Tomato

₹40

Add to Cart

Output 2: Typing "a" shows matching suggestions (Apple, Banana, Rice, Bread, Tomato) and products

Grocery Store

Apple

Banana

Bread

Tomato

Apple

₹120

Add to Cart

Banana

₹60

Add to Cart

Bread

₹45

Add to Cart

Tomato

₹40

Add to Cart

This page says
Apple added to cart

OK

Output 3: Clicking Add to Cart shows which item was added

Grocery Store

No suggestions found.

Output 4: No match shows "No suggestions found."

Result

Suggestions and product cards updated as the user typed. Add to Cart named the selected item, and a non-matching term showed the no-results message.

Scenario 7 – Netflix-style Subscription Platform

Aim

To create a subscription page offering Basic, Standard, and Premium plans and simulate payment using UPI, Card, or Net Banking.

Algorithm

1. Start the program and create a heading for the subscription service.
2. Define the available plans with their names, prices, and features.
3. Create a drop-down list so the user can select a subscription plan.
4. Create a second drop-down list for UPI, Card, and Net Banking.
5. Add a Pay Now button and an area to display the result.
6. When the button is clicked, call a JavaScript payment function.
7. Read the selected subscription plan and payment method.
8. Display a simulated payment-processing or confirmation message.
9. Show the selected plan, payment method, and plan price in the result area.
10. Clearly treat the result as a demonstration rather than a real transaction.
11. Test the page using each subscription plan and payment method.

Source Code

```
<!DOCTYPE html>
<html><body>
<h2>Subscription Plans</h2>
<label>Choose plan:
<select id="plan">
<option value="199">Basic - ₹199</option>
<option value="499">Standard - ₹499</option>
<option value="799">Premium - ₹799</option>
</select></label>
<label>Payment method:
<select
id="payment"><option>UPI</option><option>Card</option
>
<option>Net Banking</option></select></label>
<br><br><button onclick="pay()">Pay Now</button>
<div id="result"></div>
<script>
function pay(){
const plan=document.getElementById("plan");
const payment=document.getElementById("payment");
const price=plan.value;
document.getElementById("result").innerHTML=
"<h3>Demo payment successful</h3>Plan:
"+plan.options[plan.selectedIndex].text+
"<br>Amount: ₹"+price+"<br>Payment Method:
"+payment.value+
"<br><small>This is a simulation; no money was
charged.</small>";
}
</script></body></html>
```

Output

Subscription Plans

Choose plan: Payment method:

Output 1: Subscription page with plan and payment dropdowns

Subscription Plans

Choose plan: Payment method:

Demo payment successful

Plan: Basic - ₹199

Amount: ₹199

Payment Method: UPI

This is a simulation; no money was charged.

Output: Plan price Rs.199 with UPI - simulated payment result

Subscription Plans

Choose plan: Payment method:

Demo payment successful

Plan: Standard - ₹499

Amount: ₹499

Payment Method: Card

This is a simulation; no money was charged.

Output: Plan price Rs.499 with Card - simulated payment result

Subscription Plans

Choose plan: Payment method:

Demo payment successful

Plan: Premium - ₹799

Amount: ₹799

Payment Method: Net Banking

This is a simulation; no money was charged.

Output: Plan price Rs.799 with Net Banking - simulated payment result

Result

For each plan and payment method, the page displayed the selected plan, amount and payment method with a message that the payment is only a simulation.

Scenario 8 – IRCTC-style Ticket Portal

Aim

To implement a train-ticket payment demonstration that generates an order ID, records amount and date/time, and displays a simulated success or failure.

Algorithm

1. Start the program and create a train-ticket payment page.
2. Add a numeric input field for the ticket amount.
3. Add a button that starts the payment simulation.
4. When the button is clicked, read the amount entered by the user.
5. Validate that the amount is present and greater than zero.
6. Generate a unique-looking order ID using a prefix and a random number.
7. Use the JavaScript Date object to obtain the current date and time.
8. Simulate a payment result by selecting success or failure for demonstration purposes.

9. If payment succeeds, display the order ID, amount, date/time, and success status.
10. If payment fails, display a failure message and invite the user to try again.
11. Test the program with valid amounts and invalid or empty input.
12. Do not use this example for real payments; it does not connect to a payment gateway.

Source Code

```
<!DOCTYPE html>
<html><body>
<h2>Train Ticket Payment (Demo)</h2>
<input id="amount" type="number" min="1"
placeholder="Amount in rupees">
<button onclick="pay()">Make Payment</button>
<div id="result"></div>
<script>
function pay(){
const
amount=Number(document.getElementById("amount").value
);
const result=document.getElementById("result");
if(!Number.isFinite(amount)||amount<=0){
result.textContent="Enter an amount greater than
zero.";return;
}
const order="ORD"+Date.now().toString().slice(-8);
const date=new Date().toLocaleString();
if(Math.random()>0.3){
result.innerHTML="<h3>Demo Payment
Successful</h3>Order ID: "+order+
"<br>Amount: ₹"+amount.toFixed(2)+"<br>Date/Time:
"+date;
}else{
result.innerHTML="<h3>Demo Payment Failed</h3>Please
try again.";
}
}
</script></body></html>
```

Output

Train Ticket Payment (Demo)

Amount in rupees	Make Payment
---	---

Output 1: Train ticket payment page

Train Ticket Payment (Demo)

Amount in rupees	Make Payment
---	---

Enter an amount greater than zero.

Output 2: Empty amount shows a validation message

Train Ticket Payment (Demo)

1250	Make Payment
-----------------------------------	---

Demo Payment Successful

Order ID: ORD58946329

Amount: ₹1250.00

Date/Time: 10/9/2026, 8:45:46 PM

Output 3: Valid amount (1250) - simulated successful payment with Order ID and date/time

Train Ticket Payment (Demo)

Demo Payment Failed

Please try again.

Output 4: Simulated failed payment asks the user to try again

Result

Empty input was rejected. A valid amount produced a simulated result: success showed order ID, amount and date/time; failure asked the user to try again. (Success/failure is random, forced here for demonstration.)

Scenario 9 – BigBasket-style Grocery Cart

Aim

To develop a grocery shopping cart that supports adding and removing products, updating quantities, calculating the total, and simulating checkout.

Algorithm

1. Start the program and define grocery products with their names and prices.
2. Create a product display area and show each product with an Add button.
3. Create an empty cart array to store selected products and their quantities.
4. When Add is clicked, check whether the product is already in the cart.
5. If it exists, increase its quantity; otherwise, add it with quantity one.
6. Create a cart display function that clears and redraws the current cart.
7. For every cart item, display the name, unit price, quantity field, and Remove button.

8. When the quantity changes, validate it and update the selected item's quantity.
9. When Remove is clicked, delete the selected item from the cart.
10. Calculate the total by adding each item's price multiplied by its quantity.
11. Update the total amount whenever the cart changes.
12. When Checkout is clicked, check whether the cart is empty and then display a mock payment confirmation.
13. Test adding the same product multiple times, changing quantities, removing items, and checkout.

Source Code

```
<!DOCTYPE html>
<html><body>
<h2>Grocery Cart</h2>
<div id="products"></div><h2>Cart</h2>
<div id="cart"></div><h3>Total: ₹<span
id="total">0.00</span></h3>
<button onclick="checkout()">Checkout</button>
<script>
const
products=[[ "Rice",80],[ "Milk",50],[ "Apple",120],[ "Bre
ad",45],[ "Sugar",55]];
let cart=[];
document.getElementById("products").innerHTML=product
s.map((p,i)=>
`<p>${p[0]} - ₹${p[1]} <button
onclick="add(${i})">Add</button></p>`).join("");
function add(i){
const found=cart.find(x=>x.index===i);
if(found) found.qty++; else
cart.push({index:i,name:products[i][0],price:products
[i][1],qty:1});
show();
}
function changeQty(i,value){
const qty=parseInt(value,10);
if(!Number.isInteger(qty)||qty<1){cart[i].qty=1;}else
{cart[i].qty=qty;}
show();
}
function removeItem(i){cart.splice(i,1);show();}
function show(){
document.getElementById("cart").innerHTML=cart.map((p
,i)=>
```

```
`<p>${p.name} - ₹${p.price} each
<input type="number" min="1" value="${p.qty}"
onchange="changeQty(${i},this.value)">
<button
onclick="removeItem(${i})">Remove</button></p>`).join
("") || "Your cart is empty.";
const
total=cart.reduce((sum,p)=>sum+p.price*p.qty,0);
document.getElementById("total").textContent=total.to
Fixed(2);
}
function checkout(){
if(cart.length===0){alert("Your cart is
empty.");return;}
alert("Demo checkout complete. Total:
₹"+document.getElementById("total").textContent);
}
show();
</script></body></html>
```

Output

Grocery Cart

- Rice - ₹80 Add
- Milk - ₹50 Add
- Apple - ₹120 Add
- Bread - ₹45 Add
- Sugar - ₹55 Add

Cart

Your cart is empty.

Total: ₹0.00

Checkout

Output 1: Product list with empty cart (Total 0.00)

Grocery Cart

- Rice - ₹80 Add
- Milk - ₹50 Add
- Apple - ₹120 Add
- Bread - ₹45 Add
- Sugar - ₹55 Add

Cart

Rice - ₹80 each Remove

Apple - ₹120 each Remove

Total: ₹280.00

Checkout

Output 2: Rice added twice (quantity 2) and Apple once - total Rs.280

Grocery Cart

Rice - ₹80

Milk - ₹50

Apple - ₹120

Bread - ₹45

Sugar - ₹55

Cart

Rice - ₹80 each

Apple - ₹120 each

Total: ₹360.00

Output 3: Quantity of Rice changed to 3 - total updates to Rs.360

Grocery Cart

Rice - ₹80

Milk - ₹50

Apple - ₹120

Bread - ₹45

Sugar - ₹55

Cart

Rice - ₹80 each

Total: ₹240.00

Output 4: Apple removed - total updates to Rs.240

Grocery Cart

Rice - ₹80

Milk - ₹50

Apple - ₹120

Bread - ₹45

Sugar - ₹55

Cart

Rice - ₹80 each

Total: ₹240.00

This page says
Demo checkout complete. Total: ₹240.00

Output 5: Checkout shows the demo confirmation with total

Grocery Cart

Rice - ₹80

Milk - ₹50

Apple - ₹120

Bread - ₹45

Sugar - ₹55

Cart

Your cart is empty.

Total: ₹0.00

This page says
Your cart is empty.

Output 6: Checkout with an empty cart shows "Your cart is empty."

Result

Adding the same product increased its quantity, quantity edits and removals updated the total, and checkout showed a demo confirmation or an empty-cart message.

Scenario 10 – Udemy-style E-Learning Platform

Aim

To create an online course catalog with at least five courses, enrollment functionality, and a course-progress tracker.

Algorithm

1. Start the program and create an HTML page for the course catalog.
2. Prepare course data containing each course title, instructor, and price.
3. Display at least five course cards from the course data.
4. Include an Enroll button on every course card.
5. When a learner selects Enroll, store the selected course as the active course.
6. Display the active course title and a list of course modules.
7. Provide a completion control for each module.
8. Track completed modules without counting the same module more than once.
9. Calculate progress as completed modules divided by total modules multiplied by 100.
10. Update the progress percentage and completion status whenever a module is marked complete.
11. Test enrollment in different courses and completion of individual modules.

Source Code

```
<!DOCTYPE html>
<html><body>
<h2>Online Courses</h2><div id="courses"></div>
<div id="learning"></div>
<script>
const courses=[
["Python Programming","John","₹999"],
["Web Development","David","₹799"],
["Machine Learning","Priya","₹1299"],
["Java Programming","Arun","₹899"],
["UI/UX Design","Sara","₹699"]
];
let activeCourse="", completed=new Set();
document.getElementById("courses").innerHTML=courses.
map((c,i)=>`
<div><h3>${c[0]}</h3><p>Instructor:
${c[1]}</p><p>Price: ${c[2]}</p>
<button
onclick="enroll(${i})">Enroll</button></div>`).join("
");
function enroll(i){
activeCourse=courses[i][0];completed=new Set();
document.getElementById("learning").innerHTML="<h2>"+
activeCourse+"</h2>"+
[1,2,3,4].map(n=>`<p>Module ${n} <button
onclick="complete(${n})">Mark
Complete</button></p>`).join("")+
'<p id="progress">Progress: 0%</p>';
}
function complete(n){
completed.add(n);
document.getElementById("progress").textContent="Prog
ress: "+(completed.size/4*100)+"%";
```

```
}  
</script></body></html>
```

Output

Online Courses

Python Programming

Instructor: John

Price: ₹999

Enroll

Web Development

Instructor: David

Price: ₹799

Enroll

Machine Learning

Instructor: Priya

Price: ₹1299

Enroll

Java Programming

Instructor: Arun

Price: ₹899

Enroll

UI/UX Design

Instructor: Sara

Price: ₹699

Enroll

Output 1: Course catalog with five courses and Enroll buttons

Online Courses

Python Programming

Instructor: John

Price: ₹999

Enroll

Web Development

Instructor: David

Price: ₹799

Enroll

Machine Learning

Instructor: Priya

Price: ₹1299

Enroll

Java Programming

Instructor: Arun

Price: ₹899

Enroll

UI/UX Design

Instructor: Sara

Price: ₹699

Enroll

Machine Learning

Module 1 Mark Complete

Module 2 Mark Complete

Module 3 Mark Complete

Module 4 Mark Complete

Progress: 0%

Output 2: Enrolled in Machine Learning - modules shown, progress 0%

Online Courses

Python Programming

Instructor: John

Price: ₹999

Web Development

Instructor: David

Price: ₹799

Machine Learning

Instructor: Priya

Price: ₹1299

Java Programming

Instructor: Arun

Price: ₹899

UI/UX Design

Instructor: Sara

Price: ₹699

Machine Learning

Module 1

Module 2

Module 3

Module 4

Progress: 25%

Output 3: Module 1 completed - progress 25%

Online Courses

Python Programming

Instructor: John

Price: ₹999

Web Development

Instructor: David

Price: ₹799

Machine Learning

Instructor: Priya

Price: ₹1299

Java Programming

Instructor: Arun

Price: ₹899

UI/UX Design

Instructor: Sara

Price: ₹699

Machine Learning

Module 1

Module 2

Module 3

Module 4

Progress: 50%

Output 4: Module 1 clicked again (not counted twice) and Module 2 completed - progress 50%

Online Courses

Python Programming

Instructor: John

Price: ₹999

Web Development

Instructor: David

Price: ₹799

Machine Learning

Instructor: Priya

Price: ₹1299

Java Programming

Instructor: Arun

Price: ₹899

UI/UX Design

Instructor: Sara

Price: ₹699

Machine Learning

Module 1

Module 2

Module 3

Module 4

Progress: 100%

Output 5: All four modules completed - progress 100%

Result

Five courses were displayed; enrolling showed the course modules, and progress increased by 25% per module. Completing the same module twice was not counted twice, reaching 100%.

Scenario 11 – Nykaa-style Beauty Store

Aim

To create a product review form that validates a star rating and comment before displaying the submitted review.

Algorithm

1. Start the program and create a product review form.
2. Add a drop-down list containing ratings from one star to five stars.
3. Add a text area where the user can enter a review comment.
4. Add a Submit Review button and an area for validation or confirmation messages.
5. When the form is submitted, prevent the browser from refreshing the page.

6. Read the selected rating and comment text.
7. Check that a rating has been selected.
8. Check that the comment contains non-whitespace characters.
9. If either field is invalid, display an error message and do not submit the review.
10. If both fields are valid, display a confirmation and the review details.
11. Test the form with no rating, an empty comment, and a valid review.

Source Code

```
<!DOCTYPE html>
<html><body>
<h2>Beauty Product Review</h2>
<form id="reviewForm">
<label>Rating:
<select id="rating" required>
<option value="">Select Rating</option>
<option value="1">★</option><option
value="2">★ ★</option>
<option value="3">★ ★ ★</option><option
value="4">★ ★ ★ ★</option>
<option value="5">★ ★ ★ ★ ★</option>
</select></label><br><br>
<textarea id="comment" placeholder="Write your
review" required></textarea><br><br>
<button type="submit">Submit Review</button>
</form>
<div id="result"></div>
<script>
document.getElementById("reviewForm").addEventListener("submit",function(event){
event.preventDefault();
const rating=document.getElementById("rating").value;
const
comment=document.getElementById("comment").value.trim
();
const result=document.getElementById("result");
if(!rating){result.textContent="Please select a
rating.";return;}
if(!comment){result.textContent="Comment cannot be
empty.";return;}
result.innerHTML="<h3>Review submitted
successfully!</h3>Rating: "+
```

```
"★".repeat(Number(rating))+"<p>" + comment.replace(/</g, "&lt;")+"</p>";
});
</script></body></html>
```

Output

Beauty Product Review

Rating:

Please select a rating.

Output 1: Submitting with no rating shows "Please select a rating."

Beauty Product Review

Rating:

Comment cannot be empty.

Output 2: Rating selected but empty comment shows "Comment cannot be empty."

Beauty Product Review

Rating: ★★★★★ ▼

Great product, very smooth on the skin.

Submit Review

Review submitted successfully!

Rating: ★★★★★

Great product, very smooth on the skin.

Output 3: Valid rating and comment - review submitted and displayed

Result

The form rejected a missing rating and an empty comment with error messages, and displayed the review with a confirmation when both were valid.

Scenario 12 – Airbnb-style Booking Site

Aim

To develop a property-booking webpage with location and price filters, date validation, booking cost calculation, and simulated payment confirmation.

Algorithm

1. Start the program and create a property search and booking page.
2. Prepare a property list containing the location and nightly price for each property.
3. Create a location search field and maximum-price field.
4. Create a container to display properties that match the filters.
5. Whenever a filter changes, read the search location and maximum price.

6. Filter properties by location text and ensure their price does not exceed the selected maximum.
7. Display the matching properties with their nightly prices.
8. Create check-in and check-out date inputs and a Book Now button.
9. When booking is requested, verify that both dates have been entered.
10. Convert the dates to date values and check that check-out occurs after check-in.
11. Calculate the number of nights between the dates and calculate the booking amount using the selected property's nightly rate.
12. Display booking details and a simulated payment confirmation; do not claim to process a real payment.
13. Test location filters, price filters, valid dates, missing dates, and invalid date ranges.

Source Code

```
<!DOCTYPE html>
<html><body>
<h2>Property Booking</h2>
<input id="location" placeholder="Search location">
<input id="maxprice" type="number" min="0"
placeholder="Maximum price per night">
<div id="properties"></div>
<h3>Booking</h3>
<label>Property:
<select id="property">
<option value="2000">Chennai - ₹2000/night</option>
<option value="2500">Pondicherry -
₹2500/night</option>
<option value="3000">Bangalore - ₹3000/night</option>
<option value="1800">Coimbatore -
₹1800/night</option>
```

```

</select></label><br><br>
<input id="checkin" type="date"><input id="checkout"
type="date">
<button onclick="book()">Book Now</button><div
id="result"></div>
<script>
const
data=[[ "Chennai",2000],[ "Pondicherry",2500],[ "Bangalo
re",3000],[ "Coimbatore",1800]];
function show(){
const
l=document.getElementById("location").value.toLowerCa
se();
const raw=document.getElementById("maxprice").value;
const max=raw===""?Infinity:Number(raw);
document.getElementById("properties").innerHTML=data.
filter(x=>
x[0].toLowerCase().includes(l)&&x[1]<=max)
.map(x=>`<p>${x[0]} - ₹${x[1]} per
night</p>`).join("") || "No properties found.";
}
document.getElementById("location").addEventListener(
"input",show);
document.getElementById("maxprice").addEventListener(
"input",show);
function book(){
const ci=document.getElementById("checkin").value;
const co=document.getElementById("checkout").value;
const result=document.getElementById("result");
if(!ci||!co){result.textContent="Enter both check-in
and check-out dates.";return;}
const a=new Date(ci+"T00:00:00"), b=new
Date(co+"T00:00:00");
if(Number.isNaN(a.getTime())||Number.isNaN(b.getTime(
))||b<=a){

```

```

result.textContent="Check-out must be after check-
in.";return;}
const days=(b-a)/86400000;
const
price=Number(document.getElementById("property").valu
e);
result.innerHTML="<h3>Demo Booking
Confirmed</h3>Nights: "+days+
"<br>Total: ₹"+(days*price)+"<br>Payment status:
Simulated only.";
}
show();
</script></body></html>

```

Output

Property Booking

Chennai - ₹2000 per night

Pondicherry - ₹2500 per night

Bangalore - ₹3000 per night

Coimbatore - ₹1800 per night

Booking

Property:

Output 1: Property list (no filters) and booking section

Property Booking

Chennai - ₹2000 per night

Booking

Property:

Output 2: Location filter "chen" shows only Chennai

Property Booking

Chennai - ₹2000 per night

Coimbatore - ₹1800 per night

Booking

Property:

Output 3: Maximum price 2000 shows Chennai and Coimbatore

Property Booking

No properties found.

Booking

Property:

Output 4: Maximum price 500 shows "No properties found."

Property Booking

Chennai - ₹2000 per night
Pondicherry - ₹2500 per night
Bangalore - ₹3000 per night
Coimbatore - ₹1800 per night

Booking

Property:

Enter both check-in and check-out dates.

Output 5: Missing dates show an error message

Property Booking

Chennai - ₹2000 per night

Pondicherry - ₹2500 per night

Bangalore - ₹3000 per night

Coimbatore - ₹1800 per night

Booking

Property: Chennai - ₹2000/night ▼

Check-out must be after check-in.

Output 6: Check-out before check-in shows an error message

Property Booking

Chennai - ₹2000 per night

Pondicherry - ₹2500 per night

Bangalore - ₹3000 per night

Coimbatore - ₹1800 per night

Booking

Property: Pondicherry - ₹2500/night ▼

Demo Booking Confirmed

Nights: 3

Total: ₹7500

Payment status: Simulated only.

Output 7: Pondicherry, 3 nights x Rs.2500 = Rs.7500 - demo booking confirmed

Result

Location and price filters narrowed the property list. Missing dates and invalid date ranges were rejected; a valid booking calculated nights x rate (3 x 2500 = Rs.7500) with a simulated confirmation.